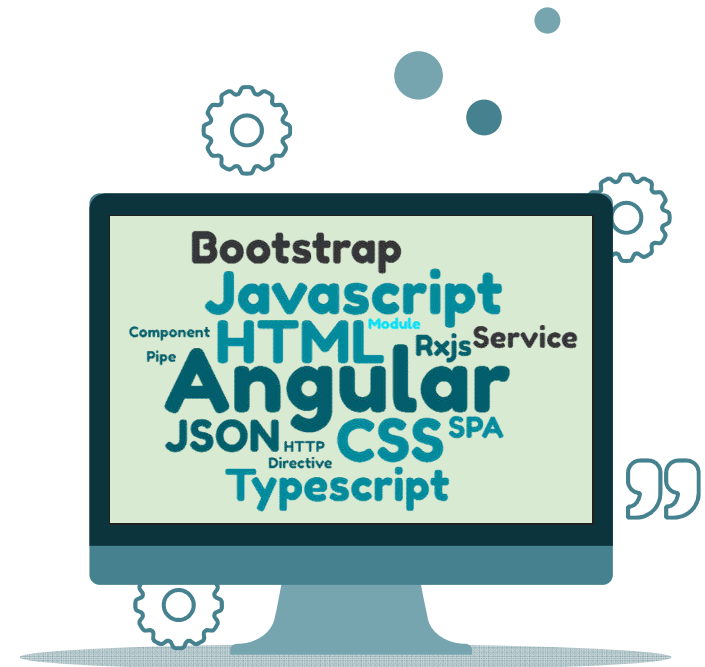


07

RxJS





Chapitre 7 RxJS

01

Observable & Subscription

02

Subject



01

Observable & Subscription



Programmation réactive

- **RxJS** (Reactive extensions for JavaScript): une librairie de programmation réactive
- Basée sur le paradigme de programmation orienté flux de données et propagation des changements
- Inspiré du **patron de conception observable (publish-subscribe)**
- Deux concepts importants :
 - **Observable** : une fonction retournant un flux de valeurs à un observateur de manière synchrone ou asynchrone. Un observable s'exécute s'il y a un observateur (observer) et une souscription (avec la méthode **subscribe()**)
 - **Observer** : un élément (objet) qui se souscrit (**subscribe()**) à un observable pour recevoir les changements et exécuter une suite de code

Pour commencer

- Créer un composant **observable**
- Le peupler un peu

```
# Création de ObservableComponent
$ ng generate component observable
```

```
app > observable > TS observable.component.ts > ...
import { Component, OnInit } from '@angular/core';

@Component({
  selector: 'app-observable',
  templateUrl: './observable.component.html',
  styleUrls: ['./observable.component.css']
})
export class ObservableComponent implements OnInit {

  status = '';
  tab: Array<number> = [];

  constructor() { }

  ngOnInit() {}
}
```

```
app > observable > <> observable.component.html > ...
<h1>éléments</h1>
<ul>
  <li *ngFor="let elt of tab">
    {{ elt }}
  </li>
</ul>
<div>{{ status }}</div>
```

localhost:4200

Welcome to first-project

éléments

Observable en bref ...

- Un **observable** est un objet qui émet des informations dans le temps
- A chaque **Observable** correspond un **Observer**
- Un **Observer** : bloc de code exécuter chaque fois qu'un **observable** émettra une information
- Un observable émet :
 - Une information
 - Une erreur
 - Ou un commit

Création d'un observable

- Dans une application Angular, vous aurez rarement à créer vous-même des observables.
- La plupart du temps, vous gérerez les flux que le framework a créés pour vous, tels que:
 - la gestion des résultats d'appels **http**,
 - l'écoute **d'événements du routeur**,
 - ou l'écoute d'événements de formulaire lors de l'utilisation du **ReactiveFormsModule**.
- Cependant, vous pouvez rencontrer des situations où il peut vous incomber de créer un flux.
- Dans les slides suivants nous verrons les principales façons dont cela pourrait se produire.

Création d'un observable : of()

- Pour créer un observable, on peut utiliser la méthode `of()` qui convertit un ensemble de paramètres en observable

```
app > observable > TS observable.component.ts > ...
import { Component, OnInit } from '@angular/core';
import { Observable, of } from 'rxjs';
-----
@Component({
  selector: 'app-observable',
  templateUrl: './observable.component.html',
  styleUrls: ['./observable.component.css']
})
export class ObservableComponent implements OnInit {

  status = '';
  tab: Array<number> = [];

  constructor() { }

  ngOnInit() {
    const observable: Observable<number> = of(1, 2, 3);
  }
}
```


Création d'un observable : from()

- On peut utiliser `from()` pour construire un observable à partir d'un tableau

```
> app > observable > TS observable.component.ts > ...
import { Component, OnInit } from '@angular/core';
import { Observable, from } from 'rxjs';
-----
@Component({
  selector: 'app-observable',
  templateUrl: './observable.component.html',
  styleUrls: ['./observable.component.css']
})
export class ObservableComponent implements OnInit {

  status = '';
  tab: Array<number> = [];

  constructor() { }

  ngOnInit() {
    const tableau = [1, 2, 3];
    const observable: Observable<number> = from(tableau);
  }
}
```

Création d'un observable : range()

- On peut utiliser `range()` pour définir un intervalle de nombre entier

```
app > observable > TS observable.component.ts > ...
import { Component, OnInit } from '@angular/core';
import { Observable, range } from 'rxjs';

@Component({
  selector: 'app-observable',
  templateUrl: './observable.component.html',
  styleUrls: ['./observable.component.css']
})
export class ObservableComponent implements OnInit {

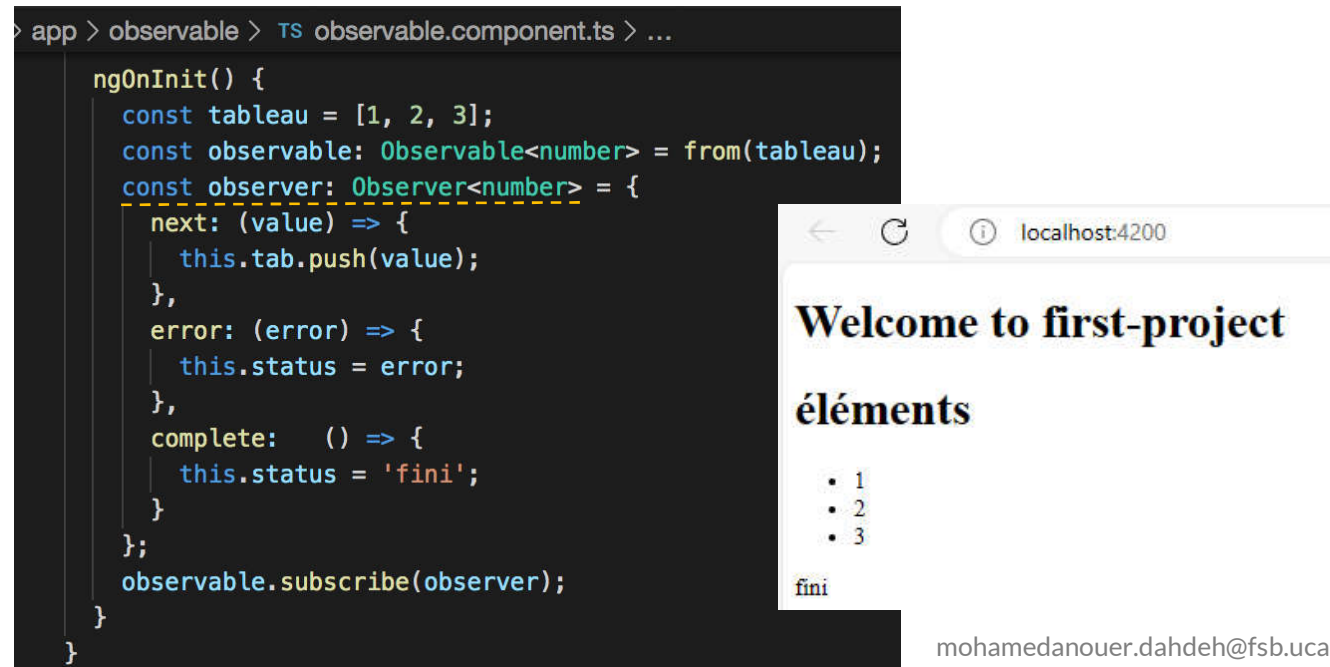
  status = '';
  tab: Array<number> = [];

  constructor() { }

  ngOnInit() {
    const observable: Observable<number> = range(1, 3)
  }
}
```

Observer

- Lorsqu'un **observateur** s'abonne à un **observable**, il peut implémenter **trois méthodes** pour spécifier ce qu'il faut faire



The image shows a code editor on the left and a web browser on the right. The code editor displays the following TypeScript code in `observable.component.ts`:

```
> app > observable > TS observable.component.ts > ...  
  
ngOnInit() {  
  const tableau = [1, 2, 3];  
  const observable: Observable<number> = from(tableau);  
  const observer: Observer<number> = {  
    next: (value) => {  
      this.tab.push(value);  
    },  
    error: (error) => {  
      this.status = error;  
    },  
    complete: () => {  
      this.status = 'fini';  
    }  
  };  
  observable.subscribe(observer);  
}
```

The web browser, running at `localhost:4200`, displays the following content:

Welcome to first-project

éléments

- 1
- 2
- 3

fini

La methode subscribe()

- La méthode **subscribe()** prend trois paramètre
 1. la 1^{ère} se déclenche à chaque fois que l'observable **émet** de **nouvelles données** (ces données sont reçues comme paramètre)
 2. la 2^{ème} se déclenche si l'observable émet **une erreur**, et reçoit cette erreur comme paramètre
 3. la 3^{ème} se déclenche lorsque l'observable **se termine**, et ne reçoit aucun paramètre.
- On peut aussi directement faire

```
//observable.subscribe(observer);
observable.subscribe({
  next: (value) => {
    this.tab.push(value);
  },
  error: (error) => {
    this.status = error;
  },
  complete: () => {
    this.status = 'fini'
  }
});
```

Observable : asynchrone

- Pour avoir une exécution asynchrone, on utilise la fonction `interval(1000)` une infinité de valeurs incrémentielles commençant de 0 : une valeur par seconde

```
ngOnInit() {  
  const observable: Observable<number>=interval(1000)  
  observable.subscribe({  
    next: (value) => {  
      this.tab.push(value);  
    },  
    error: (error) => {  
      this.status = error;  
    },  
    complete: () => {  
      this.status = 'fini'  
    }  
  });  
}
```

©MAD

Welcome to first-project

éléments

- 0

Welcome to first-project

éléments

- 0
- 1
- 2
- 3

Welcome to first-project

éléments

- 0
- 1
- 2
- 3
- 4
- 5
- 6

mohamedai

Observable : asynchrone

- On peut utiliser aussi la fonction `timer(3000, 1000)` qui envoie une infinité de valeurs incrémentielles commençant de 0 : une valeur par seconde la première valeur sera envoyée après 3 secondes.

```
ngOnInit() {  
  const observable: Observable<number>=timer(3000,1000)  
  observable.subscribe({  
    next: (value) => {  
      this.tab.push(value);  
    },  
    error: (error) => {  
      this.status = error;  
    },  
    complete: () => {  
      this.status = 'fini'  
    }  
  });  
}
```

Les deux fonctions `timer()` et `interval()` ne se terminent jamais

Observable : Unsubscribing

- Pour éviter les problèmes de mémoire, il faut penser à une souscription qui permettra de se désabonner à la destruction du composant

```
import { Component, OnDestroy, OnInit } from '@angular/core';
import { Observable, Subscription, interval } from 'rxjs';

@Component({
  selector: 'app-observable',
  templateUrl: './observable.component.html',
  styleUrls: ['./observable.component.css']
})
export class ObservableComponent implements OnInit, OnDestroy {
  status = '';
  tab: Array<number> = [];
  subscription: Subscription
  constructor() { }
  ngOnInit() {
    const observable: Observable<number> = interval(1000)
    this.subscription = observable.subscribe({
      next: (value) => {this.tab.push(value);},
      error: (error) => {this.status = error;},
      complete: () => {this.status = 'fini'}
    });
  }
  ngOnDestroy(): void {
    this.subscription.unsubscribe()
  }
}
```

Observable : Notion d'opérateur

- **Définition**

- Se place entre un observable | un subject et la souscription
- Permet **d'éditer ou filtrer les données émises** par l'observable avant qu'elles n'arrive à la souscription
- **map()** : modifie chaque valeur émise par un observable selon une fonction déclarée et ensuite l'envoie à la souscription
- **filter()** : filtre les données émises avant de les envoyer à la souscription
- **throttleTime()** : imposer un délais entre deux valeurs reçues
- **scan()** et **reduce()** : réunir toutes les valeurs émises par une observable selon une fonction définie (ex. somme de toutes la valeurs reçues)
 - **reduce** : rend le résultat
 - **scan** : rend toutes les étapes avant le résultat

Observable : Notion d'opérateur

- Pour indiquer le nombre d'élément à émettre, on utilise la méthode `pipe()` qui nous permet de faire appel à l'opérateur `take()`

```
import { take } from 'rxjs/operators';
```

```
ngOnInit() {  
  const observable: Observable<number> = interval(1000).pipe(take(4));  
  observable.subscribe(  
    (value) => { this.tab.push(value); },  
    (error) => { this.status = error; },  
    () => { this.status = 'fini'; }  
  );  
}
```

éléments

- 0
- 1
- 2
- 3

fini

`pipe()` est une fonction utilisée pour composer des opérateurs tels que `map()`, `filter()`, `take()`

Observable : Notion d'opérateur

- Combiner les opérateurs en appliquant une modification avec **map()** sur les 4 éléments

```
import { take, map } from 'rxjs/operators';
```

```
ngOnInit() {  
  const observable: Observable<number> = interval(1000).pipe(  
    take(4),  
    map(elt => elt + 3)  
  );  
  observable.subscribe(  
    (value) => { this.tab.push(value); },  
    (error) => { this.status = error; },  
    () => { this.status = 'fini'; }  
  );  
}
```

éléments

- 3
- 4
- 5
- 6

fini

Observable : Notion d'opérateur

- On peut aussi compter les éléments selon un critère avec `count()`

```
import { take, count } from 'rxjs/operators';
```

```
ngOnInit() {  
  const observable: Observable<number> = interval(1000).pipe(  
    take(10),  
    count(elt => elt % 2 === 0)  
  );  
  observable.subscribe(  
    (value) => { this.tab.push(value); },  
    (error) => { this.status = error; },  
    () => { this.status = 'fini'; }  
  );  
}
```

éléments

• 5

fini

Observable : Notion d'opérateur

- On peut sélectionner le maximum

```
import { take, max } from 'rxjs/operators';
```

```
ngOnInit() {  
  const observable: Observable<number> = interval(1000).pipe(  
    take(10),  
    max()  
  );  
  observable.subscribe(  
    (value) => { this.tab.push(value); },  
    (error) => { this.status = error; },  
    () => { this.status = 'fini'; }  
  );  
}
```

éléments

• 9

fini

Observable : Fusion

```
ngOnInit() {  
  const tableau = [1, 2, 3];  
  const observable1: Observable<number> = of(1, 2, 3);  
  const observable2: Observable<number> = of(4, 5, 6);  
  const merged = merge(  
    observable1,  
    observable2  
  );  
  merged.subscribe(  
    (value) => {  
      this.tab.push(value);  
    },  
    (error) => {  
      this.status = error;  
    },  
    () => {  
      this.status = 'fini';  
    }  
  )  
}
```

©MAD

éléments

- 1
- 2
- 3
- 4
- 5
- 6

fini

mohamedanouer.dahdeh@fsb.ucar.tn

Pipe async

- Invoquer la méthode **subscribe** sur un **Observable** et enregistrer la valeur dans une propriété du **composant** n'est pas le seul moyen d'afficher les valeurs de l'**Observable**.
- Angular fournit un **pipe** auquel l'Observable peut être transmis directement.

```
export class AppComponent {  
  counter: Observable<number>  
  
  ngOnInit(): void {  
    this.counter = interval(1000)  
  }  
}
```

```
<p>{{counter | async}}</p>
```

Pipe async: Et que faire pour les objets ?

- Pour les objets, une syntaxe alternative existe pour éviter d'utiliser de manière répétitive le pipe async pour accéder à chaque champ :

```
export class AppComponent {  
  user: Observable<User>  
  
  ngOnInit(): void {  
    this.user = interval(1000).map(_ => new User('John', 'Snow', 28))  
  }  
}  
  
interface User {  
  firstName: string  
  lastName: string  
  age: number  
}
```

map() : un operateur modifie chaque valeur émise par un observable selon une fonction déclarée, puis l'envoie à la souscription

```
<p>{{(user | async)?.firstName}}</p>  
<p>{{(user | async)?.lastName}}</p>  
<p>{{(user | async)?.age}}</p>  
  
<!-- OR -->  
  
<ng-container *ngIf="user | async as user">  
  <p>{{user.firstName}}</p>  
  <p>{{user.lastName}}</p>  
  <p>{{user.age}}</p>  
</ng-container>
```

- Aucun subscribe n'étant effectué, il n'est pas nécessaire d'unsubscribe. Le pipe async s'en charge pour nous.

02

Subject



Subject en bref ...

- Il a à la fois le rôle d'un **Observer** et d'un **observable**
- Il autorise la souscription de plusieurs observateurs

```
ngOnInit() {  
  // Création d'un subject  
  const subject = new Subject<number>();  
  
  // plusieurs observateurs peuvent s'abonner  
  subject.subscribe({  
    next : (value) => console.log("A : "+value)  
  });  
  
  subject.subscribe({  
    next : (value) => console.log("B : "+value)  
  });  
  
  subject.next(1);  
  subject.next(2);  
}
```

Angular is running in the c

A : 1

B : 1

A : 2

B : 2

Subject vs Observer

- Un **Subject** est aussi un **Observer**, il peut donc s'abonner à un **Observable**

```
ngOnInit() {  
  // Création d'un subject  
  const subject = new Subject<number>();  
  
  // plusieurs observateurs peuvent s'abonner  
  subject.subscribe({  
    next : (value) => console.log("A : "+value)  
  });  
  
  subject.subscribe({  
    next : (value) => console.log("B : "+value)  
  });  
  
  const observable : Observable<number> = from([1, 2, 3]);  
  
  observable.subscribe(subject);  
}
```

A : 1
B : 1
A : 2
B : 2
A : 3
B : 3

Exercice : Navbar

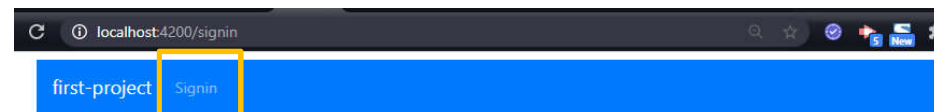
#5 BNF

Si utilisateur authentifié →

Afficher tout le menu

Sinon

Afficher seulement SignIn



Correction : Navbar

- **Etape 1:** créer un **authSubject** et fixer les informations à émettre dans **AuthService**

```
TS auth.service.ts U X
1  import { Injectable } from '@angular/core';
2  import { Subject } from 'rxjs';
3  @Injectable({
4    providedIn: 'root'
5  })
6  export class AuthService {
7    private isAuthenticated: boolean = false;
8    //Create a subject to emit the value of isAuthenticated
9    authSubject = new Subject<boolean>();
10   constructor() { }
11   //Method for subject to emit the value of isAuthenticated
12   emitAuthSubject() {this.authSubject.next(this.isAuthenticated());}
13   signIn() {this.isAuthenticated = true;
14             //Trigger the subject
15             this.emitAuthSubject()
16           }
17   signOut() { this.isAuthenticated = false;
18             //Trigger the subject
19             this.emitAuthSubject()
20           }
21   isAuthenticated(): boolean {
22     return this.isAuthenticated
23   }
24 }
```

Correction : Navbar

- **Etape 2:** souscrire **AppComponent** au **subject** et demander au **subject** d'émettre les informations

```
TS app.component.ts M X
 9  export class AppComponent implements OnInit {
10    title = 'first-project';
11    isAuth: boolean = true;
12    constructor(private authService: AuthService) { }
13    ngOnInit(): void {
14      this.authService.authSubject.subscribe({
15        next: (isAuth: boolean) => { this.isAuth = isAuth; },
16      }
17    )
18    //Trigger the transmission of information about isAuth
19    this.authService.emitAuthSubject();
20  }
21
22 }
```

Correction : Navbar

- **Etape 3:** Mettre à jour la barre de navigation dans **AppComponent** comme suit :

```
app.component.html X
src > app > app.component.html > ...
1
2 <div class="container mt-2">
3   <nav class="navbar navbar-expand-lg navbar-dark bg-primary">
4     <a class="navbar-brand" href="#">{{title}}</a>
5     <button class="navbar-toggler" type="button" data-toggle="collapse" data-target="#navbarNav"
6       aria-controls="navbarNav" aria-expanded="false" aria-label="Toggle navigation">
7       <span class="navbar-toggler-icon"></span>
8     </button>
9     <div class="collapse navbar-collapse" id="navbarNav">
10      <ul class="navbar-nav">
11        <li *ngIf="isAuth" class="nav-item"><a class="nav-link" routerLink="/" outerLinkActive="active">Home</a></li>
12        <li *ngIf="isAuth" class="nav-item"><a class="nav-link" routerLink="/about" outerLinkActive="active">About</a></li>
13        <li *ngIf="isAuth" class="nav-item"><a class="nav-link" routerLink="/contacts" outerLinkActive="active">Contacts</a>
14        <li *ngIf="!isAuth" class="nav-item"><a class="nav-link" routerLink="/signin" outerLinkActive="active">Signin</a></li>
15        <li *ngIf="isAuth" class="nav-item"><a class="nav-link" routerLink="/signin" outerLinkActive="active">Signout</a></li>
16      </ul>
17    </div>
18  </nav>
19 </div>
20 <div class="container mt-5">
21   <router-outlet></router-outlet>
22 </div>
```